

Reviewer Pack — Fourier Coefficient Sparsity and Deterministic Communication...

Entry acd684009b9d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-30 20:31:41 UTC

Fourier Coefficient Sparsity and Deterministic Communication Complexity

Entry ID: acd684009b9d **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-30 20:31:41 UTC

1. Conjecture

Field A (mathematical branch): Fourier Analysis on Boolean Functions **Field B** (complexity object): Deterministic Communication Complexity

Statement:

For any boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, the deterministic communication complexity $D(f)$ is $\Omega(1/\lambda)$, where $\lambda = \max\{|\hat{f}(S)| : S \subseteq [n]\}$ is the maximum absolute Fourier coefficient of f .

Rationale (proposer's reasoning):

Fourier coefficients capture the influence of input variables on f 's output. High λ implies sensitivity to certain variables, necessitating more communication to resolve conflicting inputs. This links harmonic analysis to communication complexity via sensitivity-based lower bounds.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c497b8c719d6404a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def fast_walsh_hadamard_transform(f: list) -> list:
    n = len(f)
    if n == 1:
        return f

    even = fast_walsh_hadamard_transform(f[:n//2])
    odd = fast_walsh_hadamard_transform(f[n//2:n])

    result = [0] * n
    for k in range(n // 2):
        result[k] = even[k] + odd[k]
        result[k + n // 2] = even[k] - odd[k]

    return result

def log_rank(matrix: list) -> int:
    m, n = len(matrix), len(matrix[0])
    rank = 0
    for i in range(m):
        pivot_row = i
        while matrix[pivot_row][i] == 0 and pivot_row < m:
            pivot_row += 1
        if pivot_row == m:
            continue

        matrix[i], matrix[pivot_row] = matrix[pivot_row], matrix[i]
        rank += 1
        for j in range(m):
            if i != j:
                factor = Fraction(matrix[j][i], matrix[i][i])
                for k in range(n):
                    matrix[j][k] -= factor * matrix[i][k]

    return rank
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 5
    c = 10 # Constant to ensure  $D(f) \geq c/\lambda$ 

    f = [random.choice([0, 1]) for _ in range(2**n)]
    fourier_coefficients = fast_walsh_hadamard_transform(f)
     $\lambda$  = max(abs(coeff) for coeff in fourier_coefficients)

    communication_matrix = [[0] * (2**n) for _ in range(2**n)]
    for x in range(2**n):
        for y in range(2**n):
            if f[x] != f[y]:
                communication_matrix[x][y] = 1

    D_f = log_rank(communication_matrix)

    conjecture_holds = D_f >= c /  $\lambda$ 
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Deterministic Communication Complexity",
        "metric_value": D_f,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}}}")
        results.append(result)

    mean = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_02a3da7d.py", line 87, in <module>
    result = run_trial(seed)
             ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_02a3da7d.py", line 68, in run_trial
    D_f = log_rank(communication_matrix)
           ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_02a3da7d.py", line 38, in log_rank
    while matrix[pivot_row][i] == 0 and pivot_row < m:
           ~~~~~^~~~~~

IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError before producing results, preventing evaluation of the conjecture. | next: Fix the test code to handle matrix indexing properly and rerun experiments

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109644
2	propose	ollama_remote	qwen3:8b	0	0	103450
3	propose	ollama_remote	qwen3:8b	0	0	110017
4	propose	ollama_remote	qwen3:8b	0	0	119816
5	preregistration	ollama_remote	qwen3:8b	0	0	24160
6	novelty	ollama_remote	qwen3:8b	0	0	21060
7	novelty	ollama_remote	qwen3:8b	0	0	15959
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13333
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10169
10	judge	ollama_remote	qwen3:8b	0	0	15245

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 542852 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/acd684009b9d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/acd684009b9d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/acd684009b9d.tar.gz` (if generated)